Muhammad Zahid
1001635161

MALLOC ASSIGNMENT REPORT

Executive Summary

This document summarizes the results from my benchmarking experiment, titled "test_file," which evaluated the performance of the standard malloc() function alongside four custom memory allocation strategies: First Fit, Best Fit, Worst Fit, and Next Fit. The primary objective of this experiment was to assess and contrast the behavior, efficiency, and performance of these allocation techniques, particularly focusing on factors such as fragmentation, heap size management, and block splitting or heap expansion.

Algorithms:

- **System Malloc:** The malloc() function, provided by standard C/C++ libraries, allocates memory from the system heap using built-in memory management techniques.
- **First Fit:** This approach allocates memory by selecting the first available block that meets the size requirement.
- **Best Fit:** In this method, the smallest block capable of satisfying the requested size is chosen for allocation.
- **Worst Fit:** This strategy allocates memory from the largest free block available, aiming to leave smaller fragments for future allocations.
- **Next Fit:** A variation of First Fit, this method starts its search for free memory blocks from the position of the most recent allocation.

Test Implementation

To evaluate the allocators, a test case was developed that performed a sequence of memory allocation and deallocation operations under controlled conditions. A timing mechanism using the clock() function was incorporated, enabling precise performance measurements. The test case included variable-sized allocations and deallocations, followed by additional steps designed to induce heap growth, block splitting, and coalescing. The collected data facilitated a detailed comparison of each custom allocator with the standard malloc().

Performance Results:

Malloc: performance rated at .001669 seconds

First Fit: performance rated at .000803 seconds

Best Fit: performance rated at .000912 seconds

Worst Fit: performance rated at .000908 seconds

Next Fit: performance rated at .001161 seconds

Muhammad Zahid
1001635161

Heap Statistics:

First Fit:
mallocs: 12
frees: 3
reuses: 2
grows: 10
splits: 0
coalesces: 0
blocks: 10
requested: 16048
max heap: 9064

Best Fit:
mallocs: 7
frees: 2
reuses: 1
grows: 6
splits: 1
coalesces: 0
blocks: 7
requested: 73626
max heap: 72636

Worst Fit:
mallocs: 7
frees: 2
reuses: 1
grows: 6
splits: 1
coalesces: 0
blocks: 7
requested: 73626
max heap: 72636

Next Fit:
mallocs: 12
frees: 3
reuses: 2
grows: 10
splits: 0
coalesces: 0

Muhammad Zahid
1001635161

blocks: 10
requested: 16048
max heap: 1024

Interpreting Results

Malloc:

The baseline malloc performance, with a rating of **0.001669 seconds**, serves as a reference for the custom allocation strategies. As a system function optimized for general memory allocation, malloc doesn't rely on specific strategies like First Fit, Best Fit, or others. Its performance is slower than the custom strategies, as it doesn't attempt to optimize for fragmentation or fit the requests into available blocks more efficiently, which could be why its performance is worse compared to other strategies.

First Fit:

The **First Fit** strategy had a performance rating of **0.000803 seconds**, making it the fastest among the strategies tested. This is likely because it allocates memory as soon as it finds the first available block that fits the request, minimizing the search time. It allocated memory 12 times and grew the heap 10 times, indicating that the heap was expanded frequently to accommodate memory requests. However, the heap grew to a maximum of 9064 bytes, which was smaller than the requested 16048 bytes, suggesting that some of the allocated memory was unused. First Fit also had low reuse and no splits or coalescing, which may point to some fragmentation over time.

Best Fit:

Best Fit performed at **0.000912 seconds**, slightly slower than First Fit but still relatively efficient. This strategy allocates memory in the smallest available block that fits the request, which can help reduce fragmentation. However, the extra overhead of searching through all blocks to find the smallest fit results in a slight performance penalty compared to First Fit. Best Fit made 7 allocations, freed 2 blocks, and grew the heap 6 times. The maximum heap size reached 72636 bytes, which was close to the total requested memory of 73626 bytes. The strategy reused one block and split another, indicating an efficient but somewhat more fragmented memory usage.

Worst Fit:

Worst Fit had a performance rating of **0.000908 seconds**, very similar to Best Fit. This strategy allocates memory in the largest available block, aiming to reduce fragmentation by leaving the largest blocks for future use. Similar to Best Fit, it allocated 7 blocks, freed 2, and grew the heap 6 times. The heap reached a maximum size of 72636 bytes, which is nearly identical to Best Fit's maximum heap size. Worst Fit also reused one block and split

one block, showing a strategy that balances between efficient allocation and managing fragmentation, but with no coalescing of free blocks, which might lead to additional fragmentation in the long term.

Next Fit:

The **Next Fit** strategy performed at **0.001161 seconds**, making it the slowest among the custom strategies. This strategy works by allocating memory from the point it last left off, aiming to reduce search times by continuing from the previous allocation. However, it showed more heap growth (10 times) and fewer block reuse opportunities (2 reuses) compared to the other strategies. The maximum heap size reached 1024 bytes, which was significantly smaller than both Best Fit and Worst Fit, indicating less efficient memory usage. Next Fit did not split blocks but also failed to coalesce free blocks, leading to potential fragmentation. The combination of slower performance and less efficient memory use suggests that Next Fit may not be as effective as the other strategies in handling larger memory requests.

AI Performance Notes

1. Did the AI assistant help?

   Yes, the AI assistant likely helped in several areas:

- **Providing Templates:** The AI provided a clear structure for common memory management strategies like Best Fit, Worst Fit, and Next Fit. This gave you a starting point to implement these features.
- **Code Completion:** The AI filled in boilerplate code and repetitive tasks (like alignment and memory block structure), which allowed you to focus on logic and specific requirements.
- **Debugging and Improvements:** Suggestions for potential issues (e.g., boundary checks, memory alignment) likely helped identify areas for improvement in my code.

2. Did it hurt?

   The AI might have hindered my learning in some aspects:

- **Abstracted Complexity:** If I relied too heavily on AI-generated code, I might not have fully understood certain low-level details of how memory allocation works.
- **Overreliance on Templates:** Predefined solutions (e.g., Best Fit or Next Fit placeholders) may lead to less engagement with the underlying logic.
- **Testing Neglect:** AI doesn't enforce testing rigor. If the testing strategy was overlooked, potential bugs (like in coalescing or splitting) may remain undetected.

Muhammad Zahid
1001635161

3. Where did the AI tool excel?

- **Clarity and Structure:** The AI provided a well-organized framework for implementing a custom memory allocator.
- The use of placeholders for allocation strategies encouraged modular development.
- **Efficiency:** It quickly generated reusable code (e.g., calloc and realloc) with minimal errors, saving time.
- **Debugging Insights:** Suggestions for coalescing logic and handling edge cases added robustness to your implementation.

4. Where did it fail?

- **Edge Case Handling:** While the AI suggested handling certain edge cases, the provided code lacked explicit mechanisms for dealing with all potential memory alignment issues or pointer mismanagement.

- **Incomplete Features:** For calloc and realloc, the AI-generated code may not handle memory resizing optimally, especially when reducing size.

- **Testing and Metrics:** The AI did not provide a comprehensive testing framework or tools to evaluate fragmentation, which is critical for memory allocators

5. Do you feel you learned more, less, or the same as if you had implemented it fully on your own?

   I feel like I would have learned more if I would have fully implemented in on my own. The reason I think that is because if i feel like the ai did a lot of things that personally would have took me a while to implement on my own. Even though the ai did a lot of things wrong, it also did a lot of things right.

6. If you learned more, what do you think you learned more of?

- **Memory Management Concepts:** Insights into managing free lists, coalescing blocks, and optimizing memory usage.

- **Implementation Strategies:** The nuances of implementing Best Fit, Worst Fit, and Next Fit.

- **Debugging and Testing:** Identifying and addressing common pitfalls in low-level programming, like boundary overflows and alignment.

Muhammad Zahid
1001635161